

COURS — INTELLIGENCE ARTIFICIELLE APPLIQUÉE

L'Architecture RAG Agentique

De la récupération statique à l'agent autonome : comment un système RAG devient un planificateur capable de raisonner, choisir ses outils et corriger ses propres erreurs.



LLM Planner



Multi-outils



Boucle ReAct



Sécurité & Guardrails

1 Du RAG classique au RAG agentique

Le **RAG traditionnel** (*Retrieval-Augmented Generation*) repose sur un flux séquentiel simple et rigide de type *Retrieve-then-Generate* : réception de la requête, interrogation d'une base vectorielle, puis génération d'une réponse unique.

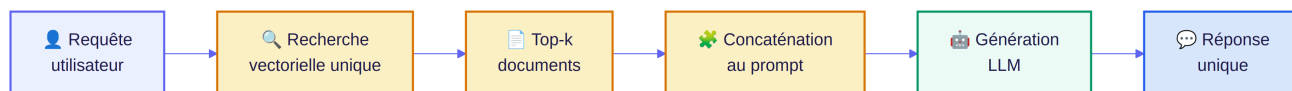


Figure 1 — Le flux linéaire et figé du RAG classique

⚠ LA LIMITE DU RAG CLASSIQUE

Le processus est **aveugle et statique**. Si la première recherche renvoie des passages non pertinents, ambigus ou incomplets, le générateur est contraint de répondre avec un contexte défaillant — ou d'halluciner.

Le changement de paradigme

Le **RAG Agentique** transforme cette chaîne linéaire en un **système autonome dynamique**. Le RAG n'est plus une étape passive, mais un **outil** à la disposition d'un agent planificateur (*LLM Planner*) capable d'évaluer la qualité de la recherche, de réécrire les requêtes, de combiner plusieurs sources ou d'itérer jusqu'à satisfaction de l'objectif.

💡 À RETENIR

On passe d'une recherche unique et passive à une **boucle de décision active** où le modèle contrôle lui-même la stratégie de récupération d'information.

Un système RAG agentique s'articule autour de trois composants majeurs interconnectés : un **cerveau planificateur**, un **inventaire d'outils** multi-sources, et une **hiérarchie de mémoire**.

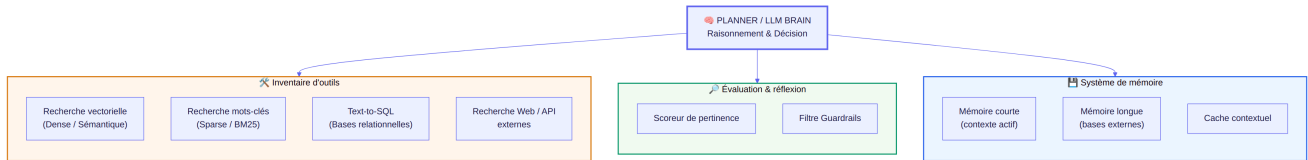


Figure 2 — Les trois piliers de l'architecture RAG agentique

A. Le cerveau planificateur (LLM Planner)

Le modèle agit comme coordinateur principal : il analyse l'intention de l'utilisateur (*Intent Classification*), évalue si une recherche est nécessaire, et détermine la séquence d'actions optimale.

B. L'inventaire d'outils multi-sources



Recherche vectorielle / sémantique

Similarité des embeddings pour capturer l'intention et le sens global.



Recherche par mots-clés (BM25)

Recherche rapide et exacte sur mots-clés, codes d'erreur ou identifiants techniques.



Text-to-SQL

Traduction du langage naturel en requêtes SQL pour interroger des bases relationnelles.



Recherche Web / API externes

Ouverture vers le web ou des services tiers quand la base interne est insuffisante.

C. La hiérarchie de mémoire

1. **Connaissances internes** — les poids pré-entraînés du modèle.
2. **Mémoire à court terme** — la fenêtre de contexte active (historique, plan, résultats d'outils, réflexions).
3. **Mémoire à long terme** — bases externes, historiques de sessions passées, bases vectorielles réinterrogées au besoin.

1. Routage dynamique (Query Routing)

Avant d'exécuter une recherche inutile ou coûteuse, un **routeur d'intention léger** examine la demande : réponse directe pour une question générale, ou orientation vers l'outil approprié (SQL pour des chiffres, vectoriel pour un contrat).

2. Reformulation de requête (Query Rewriting)

Les requêtes utilisateur sont souvent ambiguës ou dépendantes du contexte conversationnel. Un module dédié réécrit la requête pour la rendre **autonome** (*self-contained*) avant de la transmettre aux moteurs de recherche.

3. La boucle ReAct (Thought → Act → Observation)

L'agent intercale raisonnement et action à chaque étape, jusqu'à disposer de toutes les données nécessaires pour répondre.

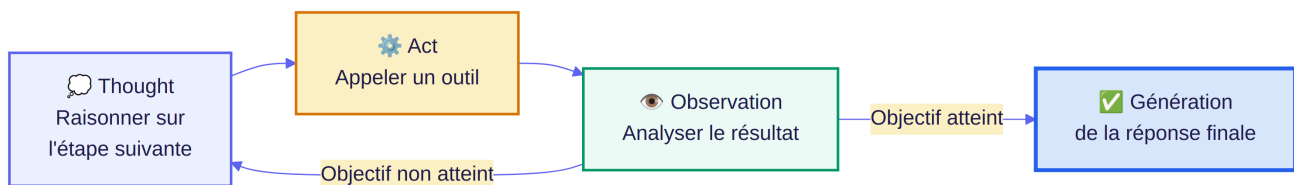


Figure 3 — La boucle itérative Thought → Act → Observation

4. Auto-correction et réflexion (Self-RAG / Corrective RAG)

Après récupération, un **module d'évaluation** vérifie la qualité des extraits :

- **Vérification d'ancrage** (*Faithfulness*) — les informations permettent-elles de répondre sans halluciner ?
- **Gestion des échecs** — si la recherche échoue, l'agent réfléchit sur son erreur, reformule ses mots-clés et retente.

Optimisations de la récupération

Pour garantir l'efficacité d'un RAG agentique en production, plusieurs techniques d'optimisation se combinent en pipeline :

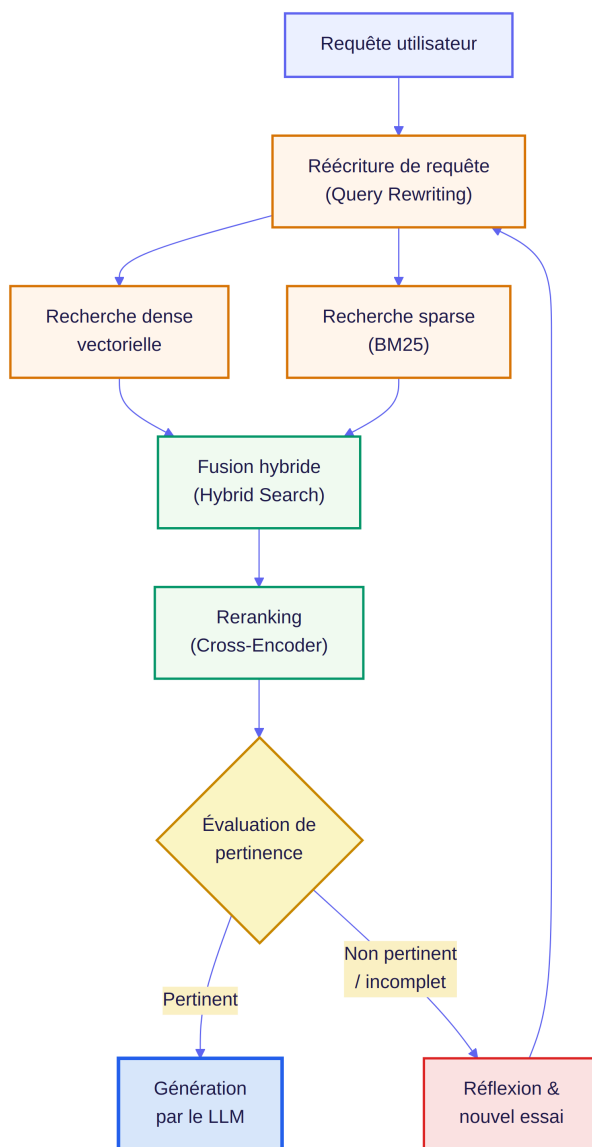


Figure 4 — Pipeline complet de récupération optimisée avec boucle de correction

4

Détail des techniques



Recherche hybride

Combinaison sémantique (vectorielle) + mots-clés (BM25) pour maximiser rappel et précision.



Reranking

Un modèle Cross-Encoder reclasse les passages les plus cruciaux en tête du prompt.



Récupération contextuelle

Ajout d'un court résumé (50-100 jetons) à l'indexation pour situer chaque chunk.



Mise en cache

Sauvegarde des réponses/vecteurs fréquents pour réduire latence et coût en jetons.

5

Sécurité et gouvernance

L'ouverture d'un agent RAG à des sources externes l'expose à l'**injection de prompt indirecte** : un attaquant peut insérer une consigne malveillante dans un document ou une page web récupérée.

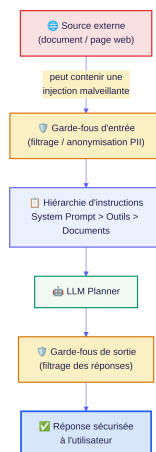


Figure 5 — Chaîne de défense contre l'injection de prompt indirecte

DEUX GARDE-FOUS ESSENTIELS

Hiérarchie d'instructions — le System Prompt prime toujours sur le contenu des documents récupérés. **Guardrails d'entrée/sortie** — anonymisation des données PII et filtrage des sorties malformées.

Caractéristique	RAG Classique (Statique)	RAG Agentique (Dynamique)
Flux de travail	Séquentiel et linéaire (Retrieve-then-Generate)	Boucle itérative avec décision dynamique
Sélection de sources	Base vectorielle unique prédéfinie	Multi-outils (Vectoriel, BM25, SQL, Web API)
Gestion des erreurs	Aucune ; répond avec les documents trouvés	Réflexion, reformulation de requêtes et retry
Précision & pertinence	Moyenne (dépendante de la première recherche)	Élevée (ajustement itératif et reranking)
Latence et coût	Faibles et prévisibles	Plus élevés (appels LLM multiples et réflexion)

💡 COMPROMIS À RETENIR

Le RAG agentique gagne en précision et en robustesse, mais au prix d'une latence et d'un coût de calcul plus élevés. Le choix dépend de la criticité de la réponse recherchée.



Exercice d'application pratique

MISE EN SITUATION

Vous concevez un assistant RH pour une entreprise. Un employé pose la question suivante : « *Combien de jours de congés me reste-t-il cette année et quelle est la procédure pour poser une demande en août ?* »

Questions :

1. Expliquez pourquoi un RAG classique risque de multiplier les erreurs ou d'échouer sur cette demande.
2. Établissez le déroulement détaillé des étapes qu'un RAG Agentique suivra pour résoudre la requête (pattern ReAct).

✓ CORRIGÉ DÉTAILLÉ

1. Limites du RAG classique — La question est hybride : elle demande une donnée personnelle structurée (solde de congés) ET un document de politique d'entreprise (procédure de demande). Un RAG classique cherchera le texte des politiques, mais ne pourra pas deviner le solde de l'employé sans interroger le système RH. Il risque donc de donner la procédure tout en **inventant** le solde restant.

2. Déroulement du RAG Agentique — voir le schéma ci-dessous.

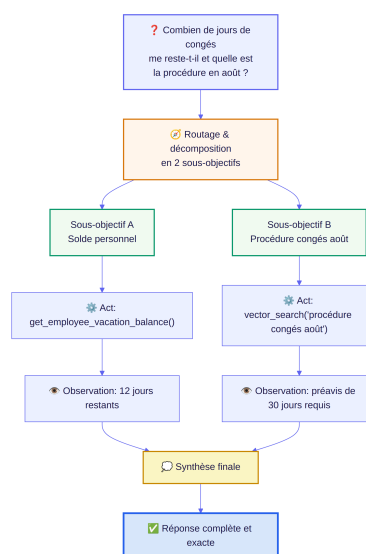


Figure 6 — Résolution d'une requête hybride par décomposition et pattern ReAct